> ***Auto-generated PDF*** *— built from* `ANALYTICS.md` *(the editable source of truth) and* `src/lib/analytics.js` *(the runtime). Regenerated by CI on every change to either file.*
>
> *Built 2026-05-15 22:11 UTC · branch* `claude/busy-pare-a438c2` *· commit* `e478397`

Analytics — Tracking Plan & GA4 Setup

This document describes what events PSPO·I Trainer fires, what they mean, and how to configure the GA4 property to surface the most useful reports.

The implementation lives in `src/lib/analytics.js`. Every event is consent-gated through CookieYes (the `cookieyes-analytics` category); if the user declines analytics, `window.gtag` never loads and all helpers no-op.

---

1. Tracking plan

Events

All events are fired client-side via `gtag('event', name, params)`.

Event	Fires when	Key parameters
`page_view`	View changes (SPA: title → home → lesson → quiz → results → …). gtag's auto page_view is **disabled**; this is the sole source.	`page_path` (`/quiz/scrum_theory`), `page_title` , `page_location`
`theme_switch`	User toggles between classic and arcade	`from` , `to`
`concept_view`	User opens a concept's lesson page	`concept_id` , `concept_label`
`quiz_start`	Any quiz starts	`mode` ∈ { `concept` , `mixed` , `mock` , `review` }, `concept_id?` , `concept_label?` , `question_count`
`quiz_complete`	Any quiz finalizes	`mode` , `concept_id?` , `concept_label?` , `score_pct` , `correct_count` , `wrong_count` , `total_questions` , `time_used_sec?` , `verdict` , `value` (=score_pct)
`quiz_pass`	Quiz finalizes with `score_pct >= 85` (PSPO I pass bar). **Fires in addition to `quiz_complete` .**	same as `quiz_complete`
`mock_exam_complete`	A mock-exam (mode= `mock`) finalizes. **Fires in addition to `quiz_complete` .**	same as `quiz_complete`
`concept_mastered`	A concept's mastery level transitions to `mastered` . Detected by diffing the mastered-set on every progress mutation.	`concept_id` , `concept_label`
`achievement_unlocked`	First time an achievement unlocks. Idempotent across sessions via localStorage. Currently: • `flawless_victory` — finished any quiz with `correct === total` • `mock_complete` — all required concepts mastered	`achievement_id` , `achievement_name`

User properties

Set on app boot and re-set whenever they change:

Property	Type	Source
`theme`	string (`classic` | `arcade`)	current theme
`concepts_mastered`	integer (0–10)	count of concepts at mastery level
`total_progress_pct`	integer	`overallProgress(progress).total`

Verdicts (the `verdict` parameter)

Computed by `verdictFor(scorePct)` in `src/lib/quiz.js` :

score_pct	verdict
≥ 95	`perfect`
≥ 85	`pass`
≥ 70	`close`
< 70	`fail`

---

2. GA4 console setup

Do this once in the GA4 property (`G-WB97T5NR8S`).

2.1 Mark these events as key events (formerly "conversions")

Admin → Events → mark as a key event:

- `quiz_pass`
- `mock_exam_complete`
- `concept_mastered`
- `achievement_unlocked`

These are the business outcomes; everything else is engagement.

2.2 Register custom event dimensions

Admin → Custom definitions → Custom dimensions → Create (scope: Event):

Dimension name	Event parameter	Description
Quiz mode	`mode`	concept / mixed / mock / review
Concept	`concept_label`	human-readable concept name
Concept ID	`concept_id`	stable id (for joins)
Verdict	`verdict`	perfect / pass / close / fail
Achievement	`achievement_name`	unlocked achievement
Theme	`theme`	classic / arcade *(if you also want it event-scoped via custom event)*

2.3 Register custom event metrics

Admin → Custom definitions → Custom metrics → Create (scope: Event):

Metric name	Event parameter	Unit
Score %	`score_pct`	Standard
Correct count	`correct_count`	Standard
Wrong count	`wrong_count`	Standard
Time used (sec)	`time_used_sec`	Standard
Total questions	`total_questions`	Standard

2.4 Register user-scoped custom dimensions

Admin → Custom definitions → Custom dimensions → Create (scope: User):

Dimension name	User property
Theme (user)	`theme`
Concepts mastered	`concepts_mastered`
Total progress %	`total_progress_pct`

2.5 Suggested data retention

Admin → Data settings → Data retention: **14 months** (max for free GA4). Reset on new activity: **On**.

2.6 Suggested IP / anonymization

GA4 anonymizes IPs by default. No additional toggles needed.

---

3. Recommended explorations

Build these once in GA4 → Explore. Each is a free-form exploration unless noted.

3.1 "Quiz mode mix"

Question: *Which quiz modes do users actually start?*

- Rows: `Quiz mode` (custom dim)
- Values: Event count for `quiz_start`, Event count for `quiz_complete`
- Add a calculated column: completion rate = `quiz_complete / quiz_start`

3.2 "Pass rate by quiz mode"

Question: *How often do users pass each mode?*

- Filter: Event name = `quiz_complete`
- Rows: `Quiz mode`, `Verdict`
- Values: Event count

- Pivot Verdict horizontally for an at-a-glance heatmap

3.3 "Hardest concepts"

Question: *Which concepts have the lowest average score on concept-mode quizzes?*

- Filter: Event name = `quiz_complete` AND `Quiz mode` = `concept`
- Rows: `Concept`
- Values: avg(`score_pct`), sum(`wrong_count`), Event count
- Sort by avg(`score_pct`) ascending

3.4 "Mock exam funnel"

Question: *How many starts convert to completes and to passes?*

- Funnel exploration
- Steps:
  1. `quiz_start` with `mode = mock`
  2. `mock_exam_complete`
  3. `quiz_pass` with `mode = mock`

3.5 "Theme preference"

Question: *Which theme do power users prefer?*

- Rows: `Theme (user)`
- Values: Active users, Average `concepts_mastered` , Event count of `quiz_complete`

3.6 "Average time-to-mastery"

Question: *How long between first quiz start and concept mastery?*

- Event count of `concept_mastered` over time
- Cross-reference with first-seen date from User Acquisition

3.7 "Engagement depth"

Question: *Do returning visitors complete more quizzes than new ones?*

- Rows: New / Returning (built-in)
- Values: avg quizzes/user, avg score, Active users

---

4. Privacy & consent

- All gtag scripts in `index.html` carry `type="text/plain"` + `data-cookieyes="cookieyes-analytics"`. They sit dormant in the DOM until CookieYes flips the type after consent.
- `window.gtag` is never defined pre-consent, so every helper in `src/lib/analytics.js` no-ops without touching the network.
- No PII is ever sent. Concept labels and question counts are the most identifying things in our payloads; no user-input text, no emails, no IPs beyond GA4's automatic (and anonymized) capture.
- Page paths are synthetic SPA routes (`/title` , `/home` , `/lesson/scrum_theory`), not URLs the user typed.

---

5. Debugging

- **Did the event fire?** Open Chrome DevTools → Network → filter `collect` . Each event POSTs to `https://www.google-analytics.com/g/collect` .
- **Real-time view:** GA4 → Reports → Real-time → DebugView. To force debug mode without the GA Debugger extension, paste this in the console after accepting analytics:

```
gtag('config', 'G-WB97T5NR8S', { debug_mode: true });
```

- **Consent dry-run:** in DevTools → Application → Cookies, delete `cookieyes-consent` and reload — you'll see the banner again.

---

6. PDF version (auto-generated)

A rendered PDF lives at **`ANALYTICS.pdf`** in the repo root — the same content as this file, formatted for printing or sharing with stakeholders who don't read markdown.

It is rebuilt automatically:

- **Locally**: `npm run docs:analytics` (uses `md-to-pdf`)
- **In CI**: `.github/workflows/build-analytics-pdf.yml` re-runs the script on every push to `main` that touches `ANALYTICS.md` , `src/lib/analytics.js` , or the build script itself, and commits the regenerated PDF back to `main` with `[skip ci]` . The PDF's header banner shows the build timestamp + commit SHA so the reader can verify it's current.

The markdown is the editable source of truth; the PDF is a derived artifact — never edit it by hand.

---

7. Future work

Out of scope for this pass, but worth considering:

- **Scroll depth on long lessons.** Fire `lesson_section_view` for each section that scrolls into view. Useful only if average reading time shows users dropping off.
- **Per-question analytics.** Currently we don't fire per-answer events — the data is too noisy and most of it lives in localStorage anyway. If a future need surfaces (e.g., "which questions are universally hardest"), an `question_answered` event with `concept_id`, `difficulty`, `correct` would do it.
- **Server-side measurement protocol** for offline events (e.g., if we ever email reminders).
- **Google Consent Mode v2 explicit signals.** Today we rely on CookieYes to gate the script entirely (a stronger guarantee than Consent Mode's "denied" state). If we ever need anonymous pings pre-consent, switch to Consent Mode v2 + remove the `data-cookieyes` block.